

Chapter 20: Exceptions

By Irina Galata

People interact with software as developers and as users, and sometimes they make mistakes. A user could, for example, input invalid data, and a developer could forget to validate it. It's important to notice and handle defects and potential weaknesses in your code in order to avoid unpredictable behavior of your app and unsatisfying experiences for its users.

Exceptions are a convenient way to detect errors in software. This concept is used in a wide variety of programming languages, including Kotlin.

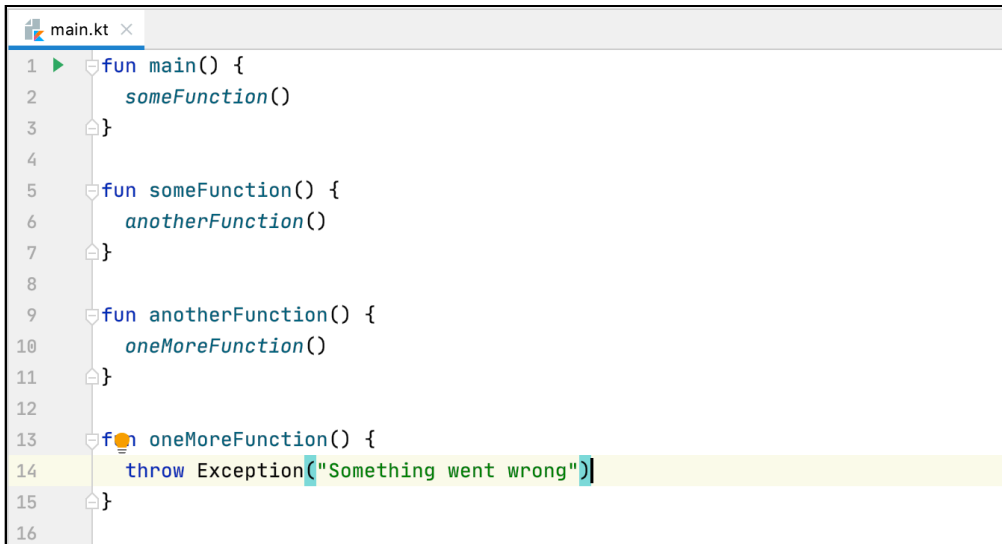
What is an exception?

An exception is primarily an event which signals that something went wrong during program execution. Exceptions are represented by the `Exception` Java class, which is a superclass of all exceptions in Java and Kotlin programs — e.g., `NullPointerException`, `IOException`, etc. Depending on their nature, these events should be **caught** at runtime or fixed to prevent the exception from happening in the first place. This chapter will discuss catching exceptions later.

Conversely, another type of critical event, an **error** — represented by `Error` and its subclasses — should not be handled but should instead be fixed, because it's the result of serious problems in a program, like inappropriate memory usage, for example.

Both `Exception` and `Error` extend `Throwable` and, therefore, could be **thrown** by the JVM or manually from code using the keyword `throw`, in order to notify the user of the code that a problem occurred. Every `Throwable` object can contain a message and a cause — another `Throwable` instance that caused this error or exception, and a **stacktrace**.

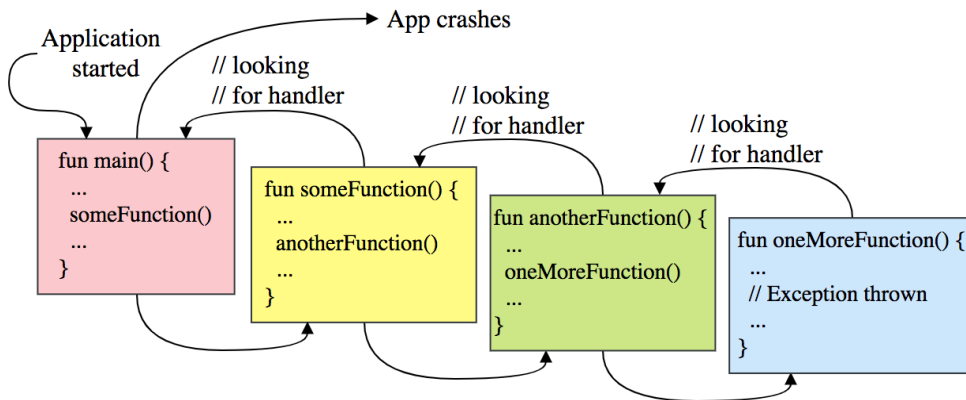
Let's see how a program behaves when an exception occurs and you don't handle it. Create a `main()` function that calls `someFunction()`, which calls `anotherFunction()`, which in turn calls `oneMoreFunction()` that throws an exception with the message "Some exception".



```
main.kt x
1 fun main() {
2     someFunction()
3 }
4
5 fun someFunction() {
6     anotherFunction()
7 }
8
9 fun anotherFunction() {
10    oneMoreFunction()
11 }
12
13 fun oneMoreFunction() {
14    throw Exception("Something went wrong")
15 }
16
```

Your program can be represented as a long chain of function invocations. When something goes wrong somewhere in `oneMoreFunction()`, an exception gets thrown and the normal execution flow interrupts.

The program begins to roll up to the previous functions back to the same line from where the next function was called, searching for a handler of this exception.



Without handling the exception, the process ends up in the entry point of your app — the `main()` function — and then the app crashes, your user sees an annoying error message.

Run the code and you will see the stacktrace of this exception in your terminal:

```

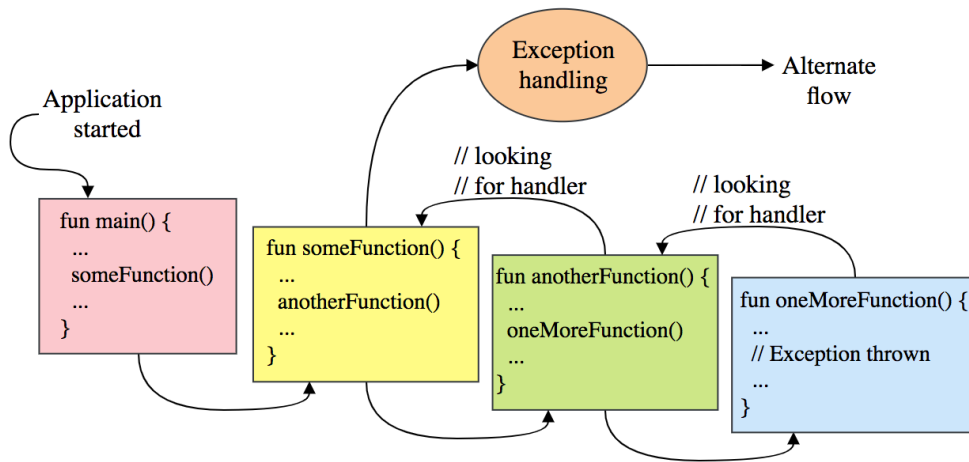
Run: MainKt x
/Library/Java/JavaVirtualMachines/jdk1.8.0_192.jdk/Contents/Home/bin/java ...
Exception in thread "main" java.lang.Exception Create breakpoint : Something went wrong
    at MainKt.oneMoreFunction(main.kt:14)
    at MainKt.anotherFunction(main.kt:10)
    at MainKt.someFunction(main.kt:6)
    at MainKt.main(main.kt:2)
    at MainKt.main(main.kt)

Process finished with exit code 1
  
```

Something went wrong at oneMoreFunction

A **stacktrace** is a detailed description of an exception that occurred in your program. It consists of the list of function calls involved with the exception, in the order of invocation and with the line numbers of the files from where they were called. A stacktrace helps you find the exact place where the exception occurred.

To prevent the app from crashing, you should **handle** an exception; you can do that in any function in the chain that led to the exception. Now look how things change if you handle an exception:



While rolling up your program, it finds a handler inside `someFunction()` and, after handling an alternate execution, the program flow re-starts and your app doesn't crash.

You'll learn how to handle errors later in this chapter. In the meantime, remove or comment out your call to `someFunction()`

For the project that follows, you will use exceptions to troubleshoot launch issues, mechanical failures and encounters with aliens!

Throwing exceptions

Imagine you're a spacecraft engineer and your main responsibility is to launch a craft for investigation of deep space. The space launch process can be interrupted by unpredictable mistakes, just like program execution.

Create a `SpaceCraft` class in the chapter starter project:

```
class SpaceCraft {  
  
    private var isConnectionAvailable: Boolean = false  
    private var isEngineInOrder: Boolean = false  
    private var fuel: Int = 0  
    var isInSpace: Boolean = false  
  
    fun launch() {  
        if (fuel < 5) {  
            sendMessageToEarth("Out of fuel. Can't take off")  
            return  
        }  
  
        if (!isEngineInOrder) {  
            sendMessageToEarth("The engine is broken. Can't take off")  
            return  
        }  
  
        if (!isConnectionAvailable) {  
            sendMessageToEarth("No connection with Earth. Can't take  
off")  
            return  
        }  
  
        sendMessageToEarth("Trying to launch...")  
        fuel -= 5  
        sendMessageToEarth("I'm in space!")  
        sendMessageToEarth("I've found some extraterrestrials")  
        isInSpace = true  
    }  
  
    fun sendMessageToEarth(message: String) {  
        println("Spacecraft to Earth: $message")  
    }  
}
```

In `launch()`, this `SpaceCraft` checks the fuel level, engine state and connection to Earth before being able to take off. If everything looks good, it sends some messages back to Earth from space. Otherwise, the method returns without sending the `SpaceCraft` to space.

All `sendMessageToEarth()` is doing is printing out the message.

Obviously, you cannot navigate your spacecraft from your room — create a `SpacePort` for this purpose:

```
object SpacePort {  
    fun investigateSpace(spaceCraft: SpaceCraft) {  
        spaceCraft.launch()  
    }  
}
```

Your spacecraft can now launch and find some aliens, but there are potential problems, e.g., a broken engine or lost connection. To check if they actually appear, create a `SpaceCraft` in your `main()` function and launch the spacecraft using the `Spaceport`:

```
val spaceCraft = SpaceCraft()  
SpacePort.investigateSpace(spaceCraft)
```

Here, you create your a `SpaceCraft` and send it through the `SpacePort` to investigate space.

Run the program. How likely are you to launch a spacecraft on the first try? Not a chance! You'll get the following message in your terminal:

```
Spacecraft to Earth: Out of fuel. Can't take off.
```

When facing any critical situation while launching, you should consider the proper way to send alerts should a problem occur. Exceptions are a good way to do so. Replace the `sendMessageToEarth()` function invocations within `launch()` with throwing an exception when a problem occurs:

```
if (fuel < 5) {  
    throw Exception("Out of fuel. Can't take off.")  
}  
  
if (!isEngineInOrder) {  
    throw Exception("The engine is broken. Can't take off.")  
}  
  
if (!isConnectionAvailable) {  
    throw Exception("No connection with Earth. Can't take off.")  
}  
// remaining launch code
```

To throw an exception, you use the `throw` keyword. Now, whenever a problem occurs, you communicate it with an exception.

Run the program again. Now the report is more detailed. You will see the following:



```
Run: MainKt x
/Library/Java/JavaVirtualMachines/jdk1.8.0_192.jdk/Contents/Home/bin/java ...
Exception in thread "main" exceptions.OutOfFuelException: Out of fuel. Can't take off
    at SpaceCraft.launch(SpaceCraft.kt:47)
    at SpacePort.investigateSpace(SpacePort.kt:38)
    at MainKt.main(main.kt:33)
    at MainKt.main(main.kt)

Process finished with exit code 1
```

Out of fuel. Can't take off at SpaceCraft.launch

Handling exceptions

As you throw exceptions, as long as you can recover from them, you should handle them. You are not obliged to handle exceptions only where you've thrown them — you can do so on any level on the stack of function calls.

Obviously, a spaceship cannot refuel or fix an engine by itself. Therefore, you should handle exceptions thrown from the spacecraft at the spaceport.

In the SpacePort object, update the investigateSpace() function to *catch* exceptions:

```
fun investigateSpace(spaceCraft: SpaceCraft) {
    try {
        spaceCraft.launch()
    } catch (exception: Exception) {
        spaceCraft.sendMessageToEarth(exception.localizedMessage)
    }
}
```

A **try-catch** expression is used to wrap a potentially problematic piece of code to avoid crashes and to handle exceptions. In the parentheses after the catch keyword, you should specify the exact type of the expected exception or its superclass. By using try-catch, an exception still gets thrown and you get notified, but the program doesn't terminate.

But remember that you have different problems and you should handle all of them separately — e.g., you won't fix the engine if the fuel tank is just empty. You need to distinguish the exceptions thrown from the spacecraft. In this case, it's not enough to use standard exceptions — you need to create custom ones.

Note: It's a bad practice to specify the parent `Exception` class in a catch statement since all thrown exceptions would be caught in this catch block. Always create separate catch blocks for every expected exception, like in the example below.

Creating custom exceptions

You already know that every child of `Exception` is an exception, too. Therefore, you just need to create subclasses of `Exception`. Add these to your project:

```
class OutOfFuelException :  
    Exception("Out of fuel. Can't take off.")
```

```
class BrokenEngineException :  
    Exception("The engine is broken. Can't take off.")
```

```
class SpaceToEarthConnectionFailedException :  
    Exception("No connection with Earth. Can't take off.")
```

Here, you're inheriting from `Exception` to make your own, specialized exceptions.

Update the `launch()` function in `SpaceCraft`:

```
if (fuel < 5) {  
    throw OutOfFuelException()  
}  
  
if (!isEngineInOrder) {  
    throw BrokenEngineException()  
}  
  
if (!isConnectionAvailable) {  
    throw SpaceToEarthConnectionFailedException()  
}  
// remaining launch code
```

Now you're using your custom exceptions rather than the generic one.

Modify the `investigateSpace()` function in `SpacePort` to look like the following:

```
fun investigateSpace(spaceCraft: SpaceCraft) {  
    try {  
        spaceCraft.launch()  
    } catch (exception: OutOfFuelException) {  
        spaceCraft.sendMessageToEarth(exception.localizedMessage)  
    } catch (exception: BrokenEngineException) {  
        spaceCraft.sendMessageToEarth(exception.localizedMessage)  
    } catch (exception: SpaceToEarthConnectionFailedException) {  
        spaceCraft.sendMessageToEarth(exception.localizedMessage)  
    }  
}
```

To catch multiple exceptions, you use several `catch` blocks. Within these blocks, you're using `localizedMessage` to get a message detailing what the exception is.

When there are several `catch` blocks, an exception is caught by the first matching block, and then program flow continues after the full `try-catch` expression.

Since you can now differentiate the exceptions, you need to take action when they occur. Add the following functions in the `SpaceCraft` class:

```
fun refuel() {  
    fuel += 5  
    sendMessageToEarth("The fuel tank is filled.")  
}  
  
fun repairEngine() {  
    isEngineInOrder = true  
    sendMessageToEarth("The engine is in order.")  
}  
  
fun fixConnection() {  
    isConnectionAvailable = true  
    sendMessageToEarth("Hello Earth! Can you hear me?")  
    sendMessageToEarth("Connection is established.")  
}  
  
fun land() {  
    sendMessageToEarth("Landing...")  
    isInSpace = false  
}
```

Now you have ways to fix any issues you might have with your spacecraft.

Move back to `SpacePort` and update the exception handling. It should now look like this:

```
try {
    spaceCraft.launch()
} catch (exception: OutOfFuelException) {
    spaceCraft.sendMessageToEarth(exception.localizedMessage)
    spaceCraft.refuel()
} catch (exception: BrokenEngineException) {
    spaceCraft.sendMessageToEarth(exception.localizedMessage)
    spaceCraft.repairEngine()
} catch (exception: SpaceToEarthConnectionFailedException) {
    spaceCraft.sendMessageToEarth(exception.localizedMessage)
    spaceCraft.fixConnection()
} finally {
    if (spaceCraft.isInSpace) {
        spaceCraft.land()
    } else {
        investigateSpace(spaceCraft)
    }
}
```

In addition to calling different functions in each `catch` block, you've also added a **finally** block to the try-catch. As opposed to the `catch` block, the code inside the `finally` block will be executed regardless of whether an exception occurs or not.

In this block, you check if your spaceship is in space or not. Depending on the result of this check, you either return the craft to Earth or relaunch it.

Run the program. You will see the following:

```
Spacecraft to Earth: Out of fuel. Can't take off
Spacecraft to Earth: The fuel tank is filled
Spacecraft to Earth: The engine is broken. Can't take off
Spacecraft to Earth: The engine is in order
Spacecraft to Earth: No connection with Earth. Can't take off
Spacecraft to Earth: Hello Earth! Can you hear me?
Spacecraft to Earth: Connection is established
Spacecraft to Earth: Trying to launch...
Spacecraft to Earth: I'm in space!
Spacecraft to Earth: I've found some extraterrestrials
Spacecraft to Earth: Landing...
```

Apparently, you've managed to overcome all of the difficulties and launched your spacecraft to space! Send greetings to the aliens for me.

Difference between Java and Kotlin exceptions

Checked exceptions

If you're familiar with Java, you may remember that there are two types of exceptions — **checked** and **unchecked**. Checked exceptions must be either handled or declared after your method signature with the `throws` keyword. Unchecked exceptions can be ignored, but then crash your app when not handled. Conversely, all exceptions in Kotlin are unchecked and, therefore, you're not forced to handle them or declare them. Your program still terminates when exceptions get thrown.

try as an expression

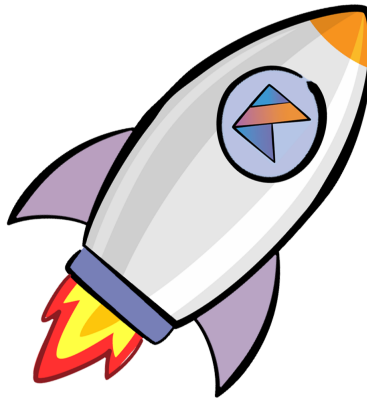
In Kotlin, the `try-catch` construction is an expression. This means that you can get a value from a `try-catch` block and can equate it to some variable:

```
val date: Date = try {  
    Date(userInput) // try to parse user input  
} catch (exception: IllegalArgumentException) {  
    Date() // otherwise use current date  
}
```

The value of the expression is equal to the last expression of the `try` block or the last expression of the `catch` block.

Challenges

1. Create a `testSetup(spaceCraft: SpaceCraft)` function in `SpacePort` which will launch your spacecraft. If it takes off successfully, this function will return `true`; if it fails, it will return `false`. Create a class `SpaceCraftException` and make it a superclass of `OutOfFuelException`, `BrokenEngineException` and `SpaceToEarthConnectionFailedException` to simplify your `try-catch-finally` expression. Don't forget to get your ship back to Earth after the test.
2. Create an `overhaul()` function in `SpaceCraft`, which will perform all necessary checks and fixes to make sure your spacecraft is ready to relaunch. Additionally, modify `investigateSpace()` in `SpacePort` so that, when an exception occurs, the spacecraft is repaired and takes off again. Use `SpaceCraftException` to simplify handling.
3. Create one more exception class called `AliensAttackException`. Throw an instance of the new exception when your spacecraft faces extraterrestrials. Handle it in `investigateSpace()`, and make sure that, after the alien confrontation, your spacecraft sends an SOS message to Earth and immediately returns to its home planet.



Key points

- **Exceptions** are the events that happen when something goes wrong in your program.
- Extend the `Exception` class or its subclasses to create custom exceptions.
- Throw an exception using the `throw` keyword.
- Do not catch the base class `Exception`, use the most specific exception class you can.
- Create custom exceptions for uncommon cases to differentiate them.
- When handling exceptions, place the code that should be executed whether an exception occurs or not in the `finally` block.
- All exceptions in Kotlin are **unchecked**.
- Don't ignore exceptions.
- Try-catch is an expression.

Where to go from here?

Exception throwing and handling can be a bit of an art. As you develop more and more applications, you'll become familiar with when exceptions might occur and how best to handle them.

In the next chapter, you'll shift from focusing on classes and object-oriented programming to instead looking at the other primary programming approach supported by Kotlin, **functional programming**.